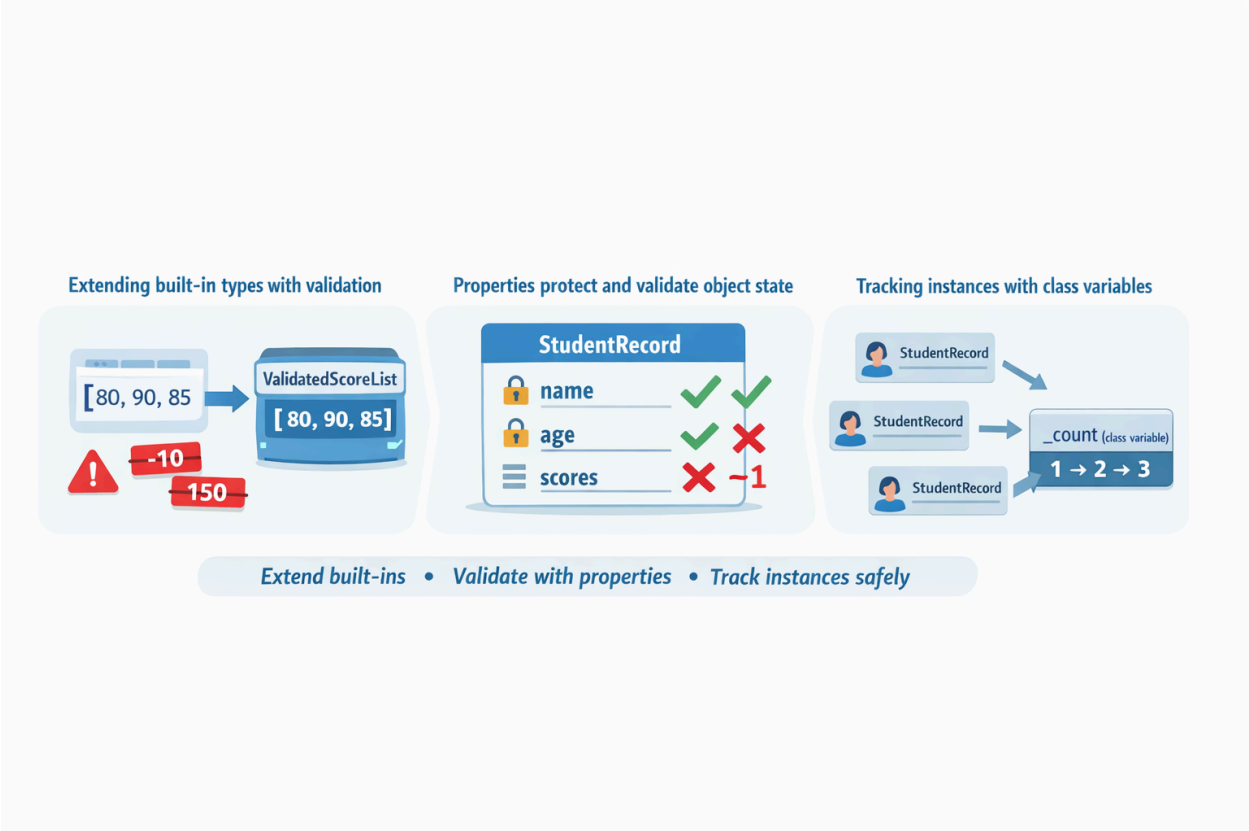
Lab 1: Extend Built-ins + Properties Validation + Instance Counting

Learning Goals

By the end of this lab, students will be able to:

- Extend a built-in type (`list`) to create a custom container
- Use `@property` to validate and protect class data
- Track how many objects are created using a class variable + classmethod



Setup

Deliverable file: `lab_extend_builtins_properties_count.py`

Run:

```
python lab_extend_builtins_properties_count.py
```

Part 0 — Start the File

Step 0: Create the file and add a title print

```
print("=== Lab: Extend Built-ins + Properties + Instance Counting ===")
```

Run the file to confirm it works.

Part 1 — Extend a Built-in Type (`list`)

Step 1: Giving your list validation logic

```
class ValidatedScoreList(list):
    """
    A list of scores that only accepts values from 0 to 100.
    """

    def _validate(self, score):
        if not isinstance(score, (int, float)):
            raise TypeError("score must be a number")
        if not 0 <= score <= 100:
            raise ValueError("score must be between 0 and 100")
```

Run the file (no output besides the title is OK).

Step 2: Override append() and extend()

Add these methods **inside the same class**:

```
  def append(self, score):
      self._validate(score)
      super().append(score)

   def extend(self, scores):
      for s in scores:
          self._validate(s)
      super().extend(scores)
```

Run the file again.

Step 3: Add average()

Add this method **inside the same class**:

```
  def average(self):
      return sum(self) / len(self) if len(self) > 0 else 0.0
```

Run the file again.

Step 4: Test ValidatedScoreList

Add this test code **below the class**:

```
print("\n--- Part 1: ValidatedScoreList ---")
scores = ValidatedScoreList([80, 90])
scores.append(70)
scores.extend([85, 95])

print("Scores:", scores)
print("Average:", scores.average())
```

Expected output (example):

```
--- Part 1: ValidatedScoreList ---
Scores: [80, 90, 70, 85, 95]
Average: 84.0
```

Part 2 — Use Properties for Validation

Step 5: Create StudentRecord shell + instance counter

```
class StudentRecord:
    """
    Student record with validated properties and a ValidatedScoreList of scores.
    Also tracks instance count.
    """
    _count = 0

    def __init__(self, name, age, scores=None):
        StudentRecord._count += 1
        self.name = name
        self.age = age
        self.scores = ValidatedScoreList(scores or [])
```

Run the file again.

Step 6: Add the name property (getter + setter)

Add this **inside StudentRecord**:

```
    @property
    def name(self):
        return self._name

    @name.setter
    def name(self, value):
        if not isinstance(value, str) or not value.strip():
            raise ValueError("name must be a non-empty string")
        self._name = value.strip()
```

Step 7: Add the age property (getter + setter)

Add this **inside StudentRecord**:

```
@property
    def age(self):
        return self._age

    @age.setter
    def age(self, value):
```

```
        if not isinstance(value, int):
            raise TypeError("age must be an integer")
        if value <= 0:
            raise ValueError("age must be a positive integer")
        self._age = value
```

Step 8: Add add_score() and instance_count()

Add this **inside StudentRecord**:

```
def add_score(self, score):
        self.scores.append(score)

    @classmethod
    def instance_count(cls):
        return cls._count
```

Step 9: Add repr for nice printing

Add this **inside StudentRecord**:

```
    def __repr__(self):
        return (
            f"StudentRecord(name='{self.name}', age={self.age}, "
            f"scores={list(self.scores)}, avg={self.scores.average():.2f})"
        )
```

Part 3 — Full Testing + Validation Errors

Step 10: Create and display StudentRecord objects

Add this test code **below the classes**:

```
print("\n--- Part 2: StudentRecord + Properties ---")
s1 = StudentRecord("Aina", 7, [80, 90])
```

```
s2 = StudentRecord("Ali", 8, [70, 75, 85])
s3 = StudentRecord("Mira", 7)

s3.add_score(88)
s3.add_score(92)

print("S1:", s1)
print("S2:", s2)
print("S3:", s3)
```

✅ Run and confirm student printing works.

Step 11: Trigger validation errors using try/except

Add this below the student printing:

```
print("\n--- Part 3: Validation Tests ---")

try:
    s1.add_score(150)
except ValueError as e:
    print("Score error:", e)

try:
    s2.age = -1
except ValueError as e:
    print("Age error:", e)

try:
    s3.name = ""
except ValueError as e:
    print("Name error:", e)
```

Expected: you should see 3 error messages printed.

Step 12: Print instance count

Add this at the end:

```
print("\n--- Instance Count ---")
print("Total StudentRecord instances:", StudentRecord.instance_count())
```

Expected:

```
Total StudentRecord instances: 3
```

Final Expected Output Summary

You should see:

- Validated score list output + average
- 3 student records printed with averages
- Validation error messages
- Final instance count

Key Takeaways

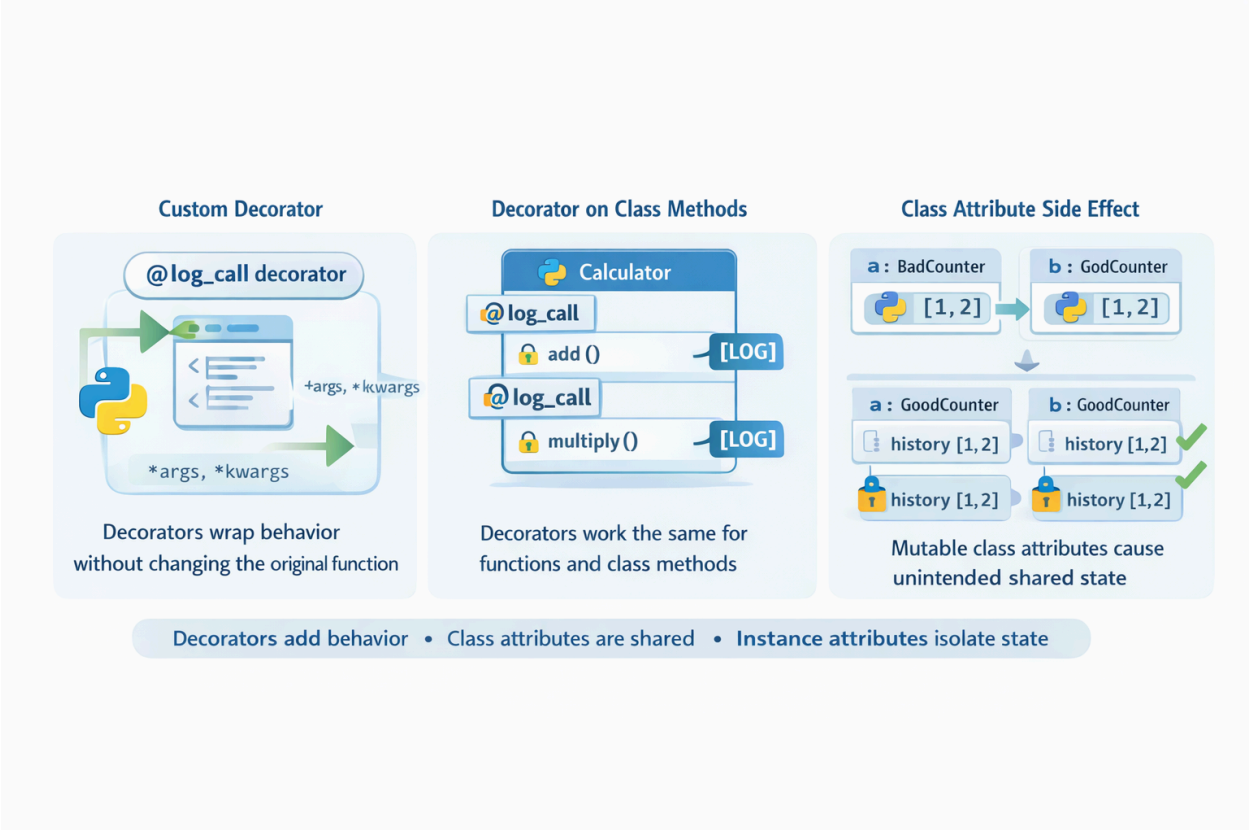
- **Built-in types can be safely extended to enforce domain rules**
  By subclassing `list`, we created a custom container that behaves like a normal list while preventing invalid data from entering the system.
- **Properties protect object state without changing how attributes are used**
  Using `@property` and setters allowed validation of `name` and `age` while keeping clean attribute access (`obj.name`, not `obj.get_name()`).
- **Validation should live as close to the data as possible**
  Placing checks inside containers and property setters ensures invalid values are blocked consistently, not just at the UI or input layer.
- **Class variables track shared information across all instances**
  The instance counter showed how class-level data is shared and how `@classmethod` provides controlled access to it.
- **Good class design leads to safer and more maintainable code**
  Encapsulation, validation, and clear responsibilities make classes easier to extend, debug, and reason about.

Lab 2: Custom Decorators & Debugging Class Attribute Side Effects

Learning Goals

By the end of this lab, students will be able to:

- Write **custom function decorators**
- Apply decorators to **class methods**
- Identify and debug **class attribute side effects**
- Understand the difference between **class attributes** and **instance attributes**



Setup

File name:

```
lab_custom_decorators_debugging.py
```

Run:

```
python lab_custom_decorators_debugging.py
```

Part 1 — Writing a Custom Function Decorator

Step 1 — Create a logging decorator

Goal: Log when a function is called and when it finishes.

```
def log_call(func):
    """
    Simple decorator that logs function calls.
    """
    def wrapper(*args, **kwargs):
        print(f"[LOG] Calling {func.__name__}")
        result = func(*args, **kwargs)
        print(f"[LOG] Finished {func.__name__}")
        return result
    return wrapper
```

This decorator wraps any function and logs messages before and after it runs, using `*args` and `**kwargs` to forward all arguments without modifying the original function.

Step 2 — Apply decorator to a function

Goal: See how decorators wrap function execution.

```
@log_call
def greet(name):
```

```
    print(f"Hello, {name}!")
```

Step 3 — Test the decorated function

```
def test_function_decorator():
    print("\n--- Function Decorator Test ---")
    greet("Aina")
```

✅ Expected output:

```
[LOG] Calling greet
Hello, Aina!
[LOG] Finished greet
```

Part 2 — Applying Decorators to Class Methods

Step 4 — Create a class using the decorator

```
class Calculator:
    @log_call
    def add(self, a, b):
        return a + b

    @log_call
    def multiply(self, a, b):
        return a * b
```

Step 5 — Test decorated methods

```
def test_method_decorator():
    print("\n--- Method Decorator Test ---")
    calc = Calculator()
    print("Add:", calc.add(2, 3))
    print("Multiply:", calc.multiply(4, 5))
```

Part 3 — Debugging Class Attribute Side Effects (Correct Example)

Objective

Demonstrate unintended shared state caused by mutating a class attribute, and help students learn how to identify and debug it.

Step 6 — Create a class with a shared class attribute (BUG)

Problematic Design

```
class BadCounter:
    """
    Demonstrates unintended shared state via a class attribute.
    """
    history = []  # ❌ shared mutable state across all instances

    def add(self, value):
        self.history.append(value)
```

📌 **Important**
`history` is defined on the **class**, not the instance — so **all objects share the same list**.

Step 7 — Observe the Side Effect

```
def test_class_attribute_side_effect():
    print("\n--- Class Attribute Side Effect ---")

    a = BadCounter()
    b = BadCounter()

    a.add(1)
    a.add(2)

    print("a.history:", a.history)
    print("b.history:", b.history)  # ❌ unexpected shared data
```

Output (What Students Will See)

```
--- Class Attribute Side Effect ---
a.history: [1, 2]
b.history: [1, 2]
```

Discussion Questions

1. **Why did `b.history` change even though we never modified it?**
2. **Where is `history` actually stored?**
3. **How many `history` lists exist in memory?**
4. **What kinds of bugs could this cause in real systems?**

Part 4 — Fixing the Side Effect

Step 8 — Move State to the Instance

```
class GoodCounter:
    """
    Correct use of instance-specific attributes.
    """

    def __init__(self):
        self.history = []  # ✅ unique per instance

    def add(self, value):
        self.history.append(value)
```

Step 9 — Verify the Fix

```
def test_fixed_counter():
    print("\n--- Fixed Counter Test ---")

    a = GoodCounter()
    b = GoodCounter()
```

```
a.add(1)
a.add(2)

print("a.history:", a.history)
print("b.history:", b.history)  # ✅ correct
```

Output

```
--- Fixed Counter Test ---
a.history: [1, 2]
b.history: []
```

Final Learning Outcome

Students should now understand that:

- Class attributes are **shared**
- Mutable class attributes are **dangerous**
- Bugs may appear *far from where the mutation occurs*
- Instance attributes prevent unintended coupling

Lab 3: Validation System + Custom Exception Hierarchy

Learning Goals

By the end of this lab, students will be able to:

- Build a reusable **validation system** (multiple rules, clear error messages)
- Design a **custom exception hierarchy** (base + specific exceptions)
- Raise and handle exceptions using **try/except/else/finally**
- Use **exception chaining** (`raise ... from e`) to preserve root cause

Setup

Create one file: **`lab_validation_custom_exceptions.py`**

Run:

```
python lab_validation_custom_exceptions.py
```

Part 0 — Start the File

Step 0: Add a title print

```
print("=== Lab: Validation System + Custom Exception Hierarchy ===")
```

Run it once to confirm the file works.

Part 1 — Build a Custom Exception Hierarchy

Step 1: Create the exception family

```
class AppError(Exception):
    """Base class for all custom application errors."""
    pass


class ValidationError(AppError):
    """Base class for validation errors."""
    def __init__(self, field: str, message: str):
        super().__init__(f"{field}: {message}")
        self.field = field
        self.message = message


class RequiredFieldError(ValidationError):
    """Raised when a required field is missing or empty."""
    pass


class RangeError(ValidationError):
    """Raised when a numeric value is out of range."""
    pass


class TypeValidationError(ValidationError):
    """Raised when a value has the wrong type."""
    pass
```

Checkpoint: You now have a clean hierarchy:

- `AppError`
    - `ValidationError`
        - `RequiredFieldError`
        - `RangeError`
        - `TypeValidationError`

Part 2 — Create Validation Rules (Reusable Validators)

Step 2: Write small validator functions

```
def require_non_empty_str(field: str, value):
    if not isinstance(value, str):
        raise TypeValidationError(field, "must be a string")
    if not value.strip():
        raise RequiredFieldError(field, "must not be empty")
    return value.strip()


def require_int_in_range(field: str, value, min_value: int, max_value: int):
    if not isinstance(value, int):
        raise TypeValidationError(field, "must be an integer")
    if not (min_value <= value <= max_value):
        raise RangeError(field, f"must be between {min_value} and {max_value}")
    return value


def require_score_list(field: str, value):
    if not isinstance(value, list):
        raise TypeValidationError(field, "must be a list of integers (0–100)")
    for i, s in enumerate(value):
        if not isinstance(s, int):
            raise TypeValidationError(field, f"score at index {i} must be an integer")
        if not (0 <= s <= 100):
            raise RangeError(field, f"score at index {i} must be between 0 and 100")
    return value
```

These functions either:

- return a cleaned/validated value, OR
- raise a **specific** validation exception.

Part 3 — Build a Validation System (Validate a “Student Record”)

Step 3: Validate a dictionary input (like API form input)

✅ Checkpoint: This is your validation “engine” for one data model.

Part 4 — Add Exception Chaining (Preserve Root Cause)

Step 4: Wrap validation with higher-level context

```
class StudentRecordBuildError(AppError):
    """Raised when building a StudentRecord fails."""
    pass


def build_student_record(data: dict) -> dict:
    """
    Higher-level builder that adds context while preserving original errors.
    """
    try:
        return validate_student_record(data)
    except ValidationError as e:
        # Add context but keep original error
        raise StudentRecordBuildError("Failed to build student record") from e
```

Now you can catch a high-level error while still seeing the original validation failure.

Part 5 — Testing (Good + Bad Cases)

Step 5: Add test runner

```
def test_cases():
    print("\n--- Test Cases ---")

    good = {"name": "Aina", "age": 7, "scores": [80, 90, 85]}
    bad_name = {"name": "", "age": 7, "scores": [80]}
    bad_age = {"name": "Ali", "age": 3, "scores": [70]}
    bad_scores = {"name": "Mira", "age": 8, "scores": [95, 150]}

    cases = [("GOOD", good), ("BAD_NAME", bad_name), ("BAD_AGE", bad_age),
("BAD_SCORES", bad_scores)]

    for label, data in cases:
        print(f"\nCase: {label}")
        try:
            record = build_student_record(data)
        except StudentRecordBuildError as e:
            print("Build error:", e)
            # show chained root cause
```

```
        print("Root cause:", repr(e.__cause__))
    else:
        print("Validated record:", record)
    finally:
        print("Done case:", label)
```

Part 6 — Main

Step 6: Call the test runner

```
def main():
    test_cases()

if __name__ == "__main__":
    main()
```

Expected Output (Example)

You should see:

- One case validates successfully
- Three cases fail with:
    - `Build error: Failed to build student record`
    - plus a `Root cause:` showing the specific validation exception (field + message)

Submission Checklist

Your `lab_validation_custom_exceptions.py` must include:

- Custom exception hierarchy (`AppError → ValidationError → specific errors`)

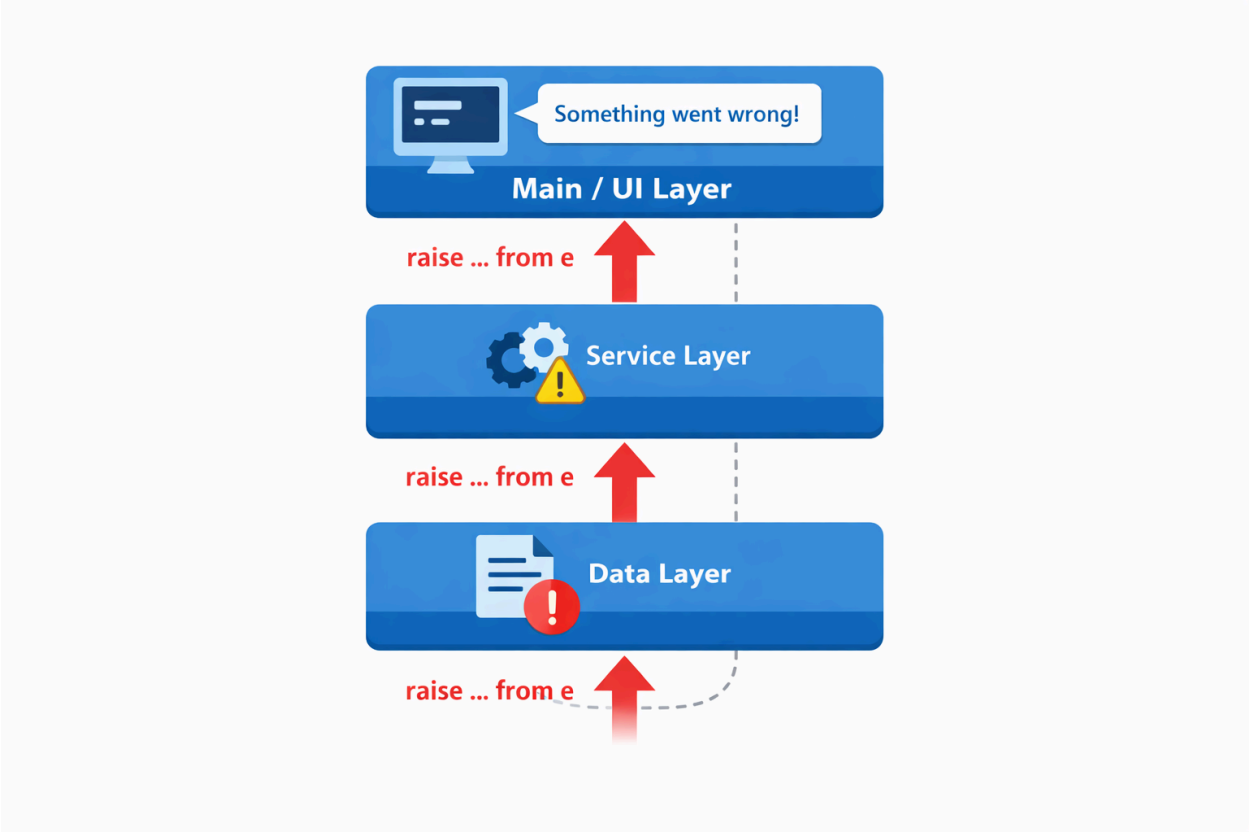
- Reusable validators (name, age range, score list)
- Validation function returning normalized data
- Exception chaining (`raise ... from e`)
- Tests using `try/except/else/finally`

Lab 4: Multi-Layer Exception Handling + Logging Errors to a File

Learning Goals

By the end of this lab, students will be able to:

- Handle exceptions across **multiple layers** (low-level → service → UI/main)
- **Translate** low-level exceptions into domain exceptions
- Use **exception chaining** (`raise ... from e`) to keep the root cause
- Log errors to a **file** with timestamps using Python's `logging`



Setup

Create one file: **lab_multilayer_exceptions_logging.py**
Run:

```
python lab_multilayer_exceptions_logging.py
```

Part 0 — Create a Logger (File Logging)

Step 0: Add this at the top of your file

```
import logging
from datetime import datetime

LOG_FILE = "app_errors.log"

logging.basicConfig(
    filename=LOG_FILE,
    level=logging.INFO,
    format="%(asctime)s | %(levelname)s | %(message)s"
)

print("=== Lab: Multi-Layer Exceptions + File Logging ===")
print(f"Logging errors to: {LOG_FILE}")
```

Checkpoint: Run the file. It should print the title and log file name.

Part 1 — Define a Custom Exception Hierarchy

Step 1: Add your domain exceptions

```
class AppError(Exception):
    """Base class for app-level exceptions."""
    pass


class DataError(AppError):
    """Raised when data access/parsing fails."""
    pass


class ServiceError(AppError):
```

```
    """Raised when business logic fails."""
    pass


class UserInputError(AppError):
    """Raised when user input is invalid."""
    pass
```

Goal: Separate errors by layer and purpose.

Part 2 — Layer 1: Data Layer (Low-Level Failures)

Step 2: Create a function that reads numbers from a “data source”

We’ll simulate reading from a file using a string.

```
def parse_numbers_from_text(text: str):
    """
    Data layer: parse comma-separated integers.
    Example input: "10,20,30"
    """
    try:
        parts = text.split(",")
        return [int(p.strip()) for p in parts]
    except Exception as e:
        # Convert low-level parsing errors into a data-layer exception
        raise DataError("Failed to parse numbers from data source") from e
```

Checkpoint idea: `parse_numbers_from_text("10,xx,30")` should raise `DataError` with a root cause.

Part 3 — Layer 2: Service Layer (Business Logic + Translation)

Step 3: Add a service that computes an average

```
def compute_average(numbers):
    """
    Service layer: compute average of a list of numbers.
    Business rule: list must not be empty.
    """
    try:
```

```
        if not numbers:
            raise ValueError("No numbers provided")
        return sum(numbers) / len(numbers)
    except Exception as e:
        raise ServiceError("Average calculation failed") from e
```

Note: The service layer doesn't care about `ValueError` directly—it exposes a clean `ServiceError`.

Part 4 — Layer 3: "UI/Main" Layer (User Handling + File Logging)

Step 4: Create a top-level function that runs the workflow

```
def run_average_workflow(user_text: str):
    """
    Main layer: orchestrates data + service layers.
    Logs errors to a file.
    """
    try:
        numbers = parse_numbers_from_text(user_text)
        avg = compute_average(numbers)
    except AppError as e:
        # Log full error including traceback
        logging.exception(f"Workflow failed for input: {user_text!r}")
        # Provide a friendly message
        print("Something went wrong. Please check your input.")
        print("Error type:", type(e).__name__)
        print("Message:", e)

        # Show root cause (from chaining)
        if e.__cause__:
            print("Root cause:", repr(e.__cause__))
    else:
        print("Numbers:", numbers)
        print("Average:", avg)
    finally:
        print("Workflow finished.\n")
```

✅ Key part: `logging.exception(...)` writes message + traceback to the log file.

Part 5 — Testing Scenarios (Good + Bad Inputs)

Step 5: Add test cases in main()

```
def main():
    print("\n--- Case 1: Good input ---")
    run_average_workflow("10,20,30")

    print("\n--- Case 2: Bad parse (non-number) ---")
    run_average_workflow("10,xx,30")

    print("\n--- Case 3: Empty numbers (business rule fail) ---")
    run_average_workflow("")

    print("\nOpen the log file to view details:")
    print(LOG_FILE)

if __name__ == "__main__":
    main()
```

Expected Results

You should see:

- Case 1 prints numbers and average
- Case 2 shows a friendly error message and logs traceback to `app_errors.log`
- Case 3 triggers a service error (empty list) and logs it too
- `Workflow finished.` prints every time (because of `finally`)

What to Check in `app_errors.log`

Open `app_errors.log` and confirm:

- timestamped error entries
- error type + message
- full traceback

- input included in the log message

Submission Checklist

Your file must include:

- 3 layers: data → service → main
- Custom exceptions by layer
- Exception chaining using `raise ... from e`
- Logging errors to a file using `logging.exception`
- Test cases with both success and failure paths

Optional Extensions (If Time Allows)

- Add a second workflow (e.g., `max`, `min`, `median`) and reuse the same layers

```
def compute_stats(numbers):
    if not numbers:
        raise ServiceError("Stats calculation failed") from ValueError("No numbers provided")

    nums = sorted(numbers)
    n = len(nums)
    mid = n // 2
    median = float(nums[mid]) if n % 2 == 1 else (nums[mid - 1] + nums[mid]) / 2.0

    return {"min": min(nums), "max": max(nums), "median": median}
```

- Add log levels: `INFO` for success, `ERROR` for failures

```
# success
```

```
logging.info(f"[STATS] Success input={user_text!r} stats={stats}")

# failure (includes traceback automatically)
logging.exception(f"[STATS] Failed for input: {user_text!r}")
```

Add a custom `UserInputError` if input is blank before parsing

Create a custom exception:

```
class UserInputError(AppError):
    pass
```

- In the **main/UI layer**, check input before parsing:

```
if not isinstance(user_text, str) or not user_text.strip():
    raise UserInputError("Input cannot be blank. Use format like: 10,20,30")
```

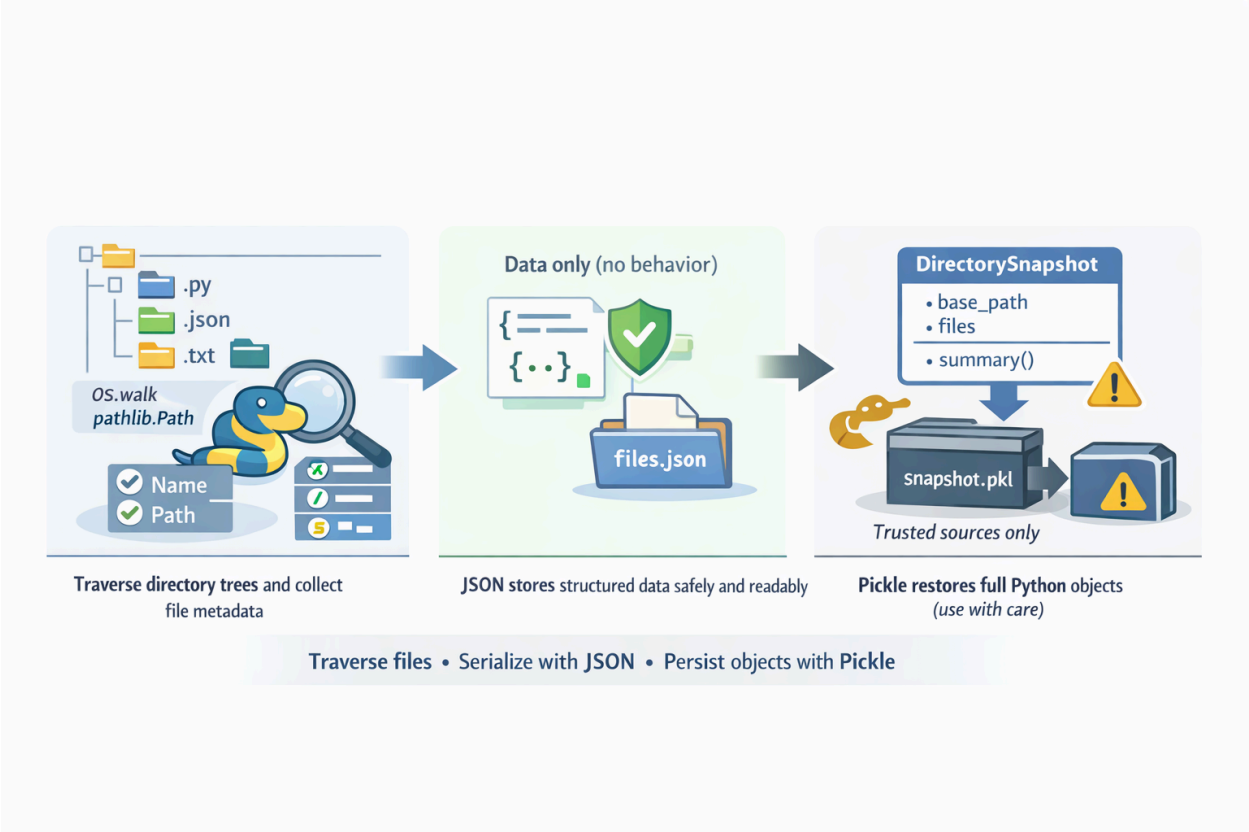
- Then catch it in the same `except AppError as e:` block (or specifically catch `UserInputError` first if you want a different message).

Lab 5 — File System Traversal + JSON + Object Persistence

Learning Goals

By the end of this lab, you will be able to:

- Traverse directory trees using `os` and `pathlib`
- Read and write JSON files safely
- Serialize and deserialize Python objects using `pickle`
- Understand when JSON vs pickle should be used



Setup

Deliverable file:

```
lab_filesystem_json_pickle.py
```

Run with:

```
python lab_filesystem_json_pickle.py
```

Part 0 — Start the File

Step 0 — Create the file and add a title

```
print("=== Lab: File System + JSON + Pickle ===")
```

Run the file to confirm it works.

Part 1 — Processing Directory Trees

Step 1 — Import required modules

```
import os
from pathlib import Path
import json
import pickle
```

Step 2 — Traverse a directory tree

Goal: Walk through a directory and collect file information.

```
def scan_directory(base_path):
    """
    Scan a directory tree and return file metadata.
    """
    files = []

    for root, dirs, filenames in os.walk(base_path):
        for name in filenames:
```

```
            full_path = os.path.join(root, name)
            files.append({
                "name": name,
                "path": full_path,
                "size": os.path.getsize(full_path)
            })

    return files
```

Step 3 — Test directory scanning

```
print("\n--- Part 1: Directory Scan ---")
base = Path(".")
file_data = scan_directory(base)

print("Files found:", len(file_data))
print("Sample:", file_data[:2])
```

Part 2 — JSON ↔ Python Conversion

Step 4 — Save Python data to JSON

```
def save_to_json(data, filename):
    with open(filename, "w", encoding="utf-8") as f:
        json.dump(data, f, indent=2)
```

Step 5 — Load JSON back into Python

```
def load_from_json(filename):
    with open(filename, "r", encoding="utf-8") as f:
        return json.load(f)
```

Step 6 — Test JSON serialization

```
print("\n--- Part 2: JSON Serialization ---")
save_to_json(file_data, "files.json")

loaded_json = load_from_json("files.json")
```

```
print("Loaded from JSON:", len(loaded_json))
```

Discussion

- JSON stores **data only**, not behavior
- JSON is human-readable and safe

Part 3 — Object Persistence with Pickle

Step 7 — Create a simple class

```
class DirectorySnapshot:
    """
    Represents a snapshot of scanned directory data.
    """
    def __init__(self, base_path, files):
        self.base_path = str(base_path)
        self.files = files

    def summary(self):
        return f"{len(self.files)} files in {self.base_path}"
```

Step 8 — Save object using pickle

```
def save_with_pickle(obj, filename):
    with open(filename, "wb") as f:
        pickle.dump(obj, f)
```

Step 9 — Load object using pickle

```
def load_with_pickle(filename):
    with open(filename, "rb") as f:
        return pickle.load(f)
```

Step 10 — Test pickle persistence

```
print("\n--- Part 3: Pickle Persistence ---")

snapshot = DirectorySnapshot(base, file_data)
print("Snapshot summary:", snapshot.summary())

save_with_pickle(snapshot, "snapshot.pkl")

restored = load_with_pickle("snapshot.pkl")
print("Restored summary:", restored.summary())
```

📌 **Important Warning**

- Pickle is **NOT SAFE** for untrusted files
- Only load pickle files you created yourself

Part 4 — Compare JSON vs Pickle

Step 11 — Reflect

Ask yourself:

- Why can JSON store lists/dicts but not objects?
- Why does pickle restore full object behavior?
- Which is safer for config files?

Final Expected Output (Example)

You should see:

- Number of files scanned
- JSON saved and loaded successfully
- Pickle object restored with methods intact

Submission Checklist

Your `lab_filesystem_json_pickle.py` must include:

- Directory traversal using `os.walk` or `pathlib`
- JSON save + load functions
- A custom class persisted with pickle
- Successful execution without errors

Optional Extensions (If Time Allows)

- Filter files by extension (e.g. `.py` only)

```
def scan_directory(base_path, ext_filter=None):
    """
    Scan a directory tree and return file metadata.
    ext_filter: e.g. ".py" (only include files ending with .py)
    """
    files = []

    for root, dirs, filenames in os.walk(base_path):
        for name in filenames:
            if ext_filter and not name.endswith(ext_filter):
                continue

            full_path = os.path.join(root, name)
            files.append({
                "name": name,
                "path": full_path,
                "size": os.path.getsize(full_path)
            })

    return files


file_data = scan_directory(Path("."), ext_filter=".py")
```

- Add timestamp to directory snapshot

```
from datetime import datetime

class DirectorySnapshot:
    def __init__(self, base_path, files):
        self.base_path = str(base_path)
        self.files = files
        self.created_at = datetime.now().isoformat(timespec="seconds")

    def summary(self):
        return f"{len(self.files)} files in {self.base_path} (created_at={self.created_at})"
```

- Compare file sizes before/after serialization

```
def file_size_bytes(filename):
    return os.path.getsize(filename)
```

```
save_to_json(file_data, "files.json")
save_with_pickle(snapshot, "snapshot.pkl")

json_size = file_size_bytes("files.json")
pkl_size = file_size_bytes("snapshot.pkl")

print("\n--- Size Comparison ---")
print("files.json size (bytes):", json_size)
print("snapshot.pkl size (bytes):", pkl_size)
```

- Add exception handling for missing files

```
def load_from_json(filename):
    try:
        with open(filename, "r", encoding="utf-8") as f:
            return json.load(f)
    except FileNotFoundError:
        print(f"[ERROR] JSON file not found: {filename}")
        return []
    except json.JSONDecodeError as e:
        print(f"[ERROR] Invalid JSON in {filename}: {e}")
        return []


def load_with_pickle(filename):
    try:
        with open(filename, "rb") as f:
            return pickle.load(f)
    except FileNotFoundError:
        print(f"[ERROR] Pickle file not found: {filename}")
        return None
    except Exception as e:
        print(f"[ERROR] Failed to load pickle {filename}: {e}")
        return None
```

```
print("\n--- Missing File Tests ---")
print(load_from_json("not_here.json"))
print(load_with_pickle("not_here.pkl"))
```

Key Takeaways

- File systems are hierarchical — traversal matters
- JSON is safe, portable, and readable
- Pickle is powerful but dangerous if misused
- Choose persistence tools based on **trust and use case**